

## Cryptographic Misuse Patterns

### The honest headline

Modern algorithms (AES, RSA, ECDH, SHA-2/3) are sound. Attacks on crypto in the real world are almost always **misuse**: wrong mode, weak key, reusing nonces, funny PRNG, or custom algorithms.

### Misuse playbook

| Pattern                         | Why it's weak                                | Classic result                                     |
|---------------------------------|----------------------------------------------|----------------------------------------------------|
| ECB mode                        | Same plaintext block → same ciphertext block | Data repeats visible in patterns (see the penguin) |
| CBC + predictable IV / reuse    | IV reuse leaks relationships                 | plaintext recovery with a few chosen pairs         |
| Short keys (40-bit, 56-bit DES) | Exhausted by dedicated hardware              | DES keys brute-forced feasibly                     |
| Nonce reuse (CTR/GCM, ChaCha)   | Stream keystream repeats                     | XOR of two plaintexts recoverable                  |
| Unpadded RSA signature schemes  | Multiplicative properties                    | forgery / Bleichenbacher-type attacks              |
| Weak PRNG: rand() seeded        | Predictable outputs                          | Predict all 'random' keys                          |
| Homegrown crypto                | No peer review                               | Almost certainly broken (rare exceptions)          |
| Hashing passwords once, fast    | No salt + fast hash                          | Rainbow/GPU crack trivial                          |

### The impossible of 'verifying' crypto with eyes

Signatures/encryption look identical to a human. That's why *operation* and *coverage* matter:

Are libraries current (OpenSSL/LibreSSL), not hand-rolled?

Are all data-at-rest/in-transit paths covered (a forgotten HTTP endpoint?)

Key management: rotation, HSMs, vault, no keys in configs

Logging: CBC/ECB/nonce audit impossible on plaintext—so approve by design, not by inspection

Good default stack for new code:

AES-256-GCM (AEAD) or ChaCha20-Poly1305 at rest/in transit

Argon2id/bcrypt/scrypt for password hashes (salted)

X25519 + AEAD for key exchange; ECDSA for signing

Learn the misuse table cold. When you audit a system and can name which pattern its crypto follows, you've already found the report's centerpiece.

---

[← Module start](#)

[Brute Force, Rainbow Tables & ...](#)